

California Lutheran

UNIVERSITY

Course 220: Advanced Programming

Fall 2026

Instructor Information:	Dr. Fahmida Hamid Associate Professor
Office Hours & Location	• Mon 11:00 am ~ 12:00 pm, • Wed 11:00 am ~ 12:00 pm, and • Thurs 11:30 am ~ 12:30 pm • Or <u>by appointment (via email)</u> <u>@ Campus Office:</u> ASCI 120
Contact Info:	CLU Phone: (805) 493-3040 CLU Email: fhamid@callutheran.edu
Class Meeting Times & Location:	TTh, 2:15pm ~ 3:55pm Room # ASCI 203
Drop Dates:	09/06/2026 [academic calendar]

A. Course Description:

Introduction of object-oriented programming and data structures. Using the Java programming language students develop skills to select appropriate programming techniques and data structures for a given problem specification. Those data structures are then implemented using the object-oriented design methodology. Topics covered include: 1) inheritance; 2) composition/aggregation; 3) interface and abstract class; 4) data-structures (multi-dimensional array, linked-list, stack, queue, hash-map); 5) event-driven programming; 6) multi-threaded programming; 7) Run-Time-Type-Identification; 8) recursion.

Emphasis will be on writing programs. Programming will be done in the Java programming language.

B. Pre-requisite Knowledge or Coursework:

MATH 241 Discrete Math **and**
CSC 210 Introduction to Programming

C. Textbook (Required):

Title: Introduction to Java Programming, Comprehensive Version, 12th Edition (or later)

Author: Daniel Liang Publisher: Pearson, 2019

Language: English

ISBN: 978-0136520238

D. Program Learning Outcomes:

The Computer Science/Computer Information Systems degree programs within the Computer Science Department address a wide spectrum of topics ranging from theoretical work to practical application. To be competitive in this broad range, we focus on foundational computer science theory, current computer technology, and application. We are committed to the development of excellence in teaching and research as a means of preparing students for a smooth transition from academia to a career in computer science or related areas. Upon the completion of CS/CIS Major, students are expected to have learned the following:

- **Technology Integration:** Students will be able to combine necessary theories, software programs, and operating systems to address real-world issues.
- **Information Technology:** Students will be able to understand and utilize current development tools to build revised or new platforms for future application.
- **Analytical/Critical Thinking:** Students will be able to analyze, understand, and modify an existing framework and to propose a new strategy within the given constraints.

E. Course Learning Outcomes:

- **Creative and Critical Thinking:** By the end of this course, students will develop creative and critical thinking skills in assignments requiring conversion of a functional specification (written in a natural language, e.g. English) into algorithm implementations through the application of various advanced programming techniques.
 - Design and implement solutions using advanced programming techniques.
 - Select and apply appropriate data structures to solve programming problems.
 - Develop multithreaded and event-driven applications.
 - Evaluate alternative programming approaches and justify design decisions.
 - Debug, test, and document Java programs.

- **Information Literacy:** By the end of this course, students will develop information literacy skills in assignments requiring understanding of what it means to create a program using advanced techniques.
 - When to deploy advanced techniques and when simple constructs suffice
 - When to write your own code and when to utilize predefined libraries

F. Class Format: This 15-week course is 100% face-to-face meetings. Course meetings consist of lectures, in class exercises, and exams.

G. Course Assignments:

Assessment is based on multiple components, each designed to evaluate different learning outcomes and skills.

- **Assignments:** Assignments assess understanding of course material and may be completed individually or in teams of up to three students. Peer discussion is permitted, but all external assistance (including third-party input) must be acknowledged in source code comments. Uncredited assistance may be treated as a violation of the University's Academic Honesty Policy.

Assignments are graded on both correctness and coding style. Unless otherwise specified, submissions must include:

- Source code files (.java)
- Screenshots demonstrating execution
- Screenshots of program output
- All submissions must be made through Canvas by 11:59 PM on the due date. Late submissions will not receive credit.
- **In-class Work:** These must be completed while in class and shown to the prof. before leaving. Students are expected to complete quizzes on the scheduled date. Makeup quizzes are granted only for documented medical reasons or approved arrangements made at least 72 hours in advance.
- **Project:** Students will complete a final project in teams of two to three members. The project provides an opportunity to apply concepts learned throughout the course to the design and implementation of a meaningful software application. Students are encouraged to pursue projects that align with their interests, academic goals, or real-world problems. Through this multi-week project, students will gain experience in project planning, software design, collaborative development, task management, version control, and the integration of multiple programming concepts into a cohesive solution.

The project must demonstrate several of the following advanced programming concepts:

- Successful application of
 - Inheritance, Interface, or Abstract Classes
 - Collection Framework
 - Exception Handling
 - File Persistence
 - GUI
 - SOLID principles
- A single Git Repository (must)
 - Each team member is expected to contribute to the project repository. Git commit history may be used when evaluating individual participation.

Project Deliverables:

- Source code
 - Git repository (link)
 - README containing setup and execution instructions
 - Design documentation (UML or architecture overview)
 - Runnable application (JAR file when applicable)
 - Short project demonstration and presentation
- **Exams:** There will be two in-class, closed-book exams (written or oral) during the semester. Exams should reflect your own understanding of the concepts.

The overall breakdown is:

Assessment	Percentage
In-class Work	25
Assignment	20
Exam	25
Project	30
Total	100

H. Course Grading Policy

Percentage	Grade	Percentage	Grade
> 97	A	79 to 81	C
94 to 97	A-	76 to 78	C-
91 to 93	B+	73 to 75	D+
88 to 90	B	70 to 72	D
85 to 87	B-	67 to 69	D-
82 to 84	C+		

I. Course Schedule:

Week	Date	Topic	Reading	Activity
01	Aug 25 Aug 27	Introduction to the concepts of Object-Oriented Programming: Abstraction, Inheritance, Polymorphism + Common Documentation Practices	Chapter 9, 10, 11	In-class Task
02	Sep 01 Sep 03	Object-Oriented Programming: Object Relationships (Association, Aggregation, Composition) + UML Diagram	Chapter 9, 10, 11, 13	In-class Task
03	Sep 08 Sep 10	Exception and Error Handling in Java, + File Operations + JavaDoc Documentation	Chapter 12	In-class Task & Assignment 01 <1-week to complete>
04	Sep 15 Sep 17	Generics + Intro to SOLID Principles	Chapter 19	In-class Task
05	Sep 22 Sep 24	Array, Array List, Linked List, Stack, Queue + Asymptotic Analysis	Chapter 20, 24, 27	In-class Task + Assignment 02 <1-week to complete>
06	Sep 29 Oct 01	Hash, Hash Map, Serialization, File I/O	Chapter 17, 21, 27	In-class Task

07	Oct 06 Oct 08	Practice/Mock Exam Day Term Exam I		In-class, closed-book
08	Oct 13 Oct 15	Graphical User Interfacing, Graphics Event - JavaFX/Swing, + Java Stream API	Chapter 14, 15, 16	Assignment 03 <1-week to complete>
09	Oct 20 Oct 22	Event-Driven Programming + Unit Testing	Chapter 14, 15, 16	In-class Task
10	Oct 27 Oct 29	Multi-Threading, + Code Refactoring	Chapter 32	In-class Task
11	Nov 03 Nov 05	Recursion	Chapter 18	In-class Task & Assignment 04 <1-week to complete>
12	Nov 10 Nov 12	Project Workshop + Version Control with Git and GitHub		class: proposal writing, solution design, work-in progress...
13	Nov 17 Nov 19	Project Workshop		Continue working on the project <must attend class> + project presentation (finals week)
14	Nov 24	Term Exam II		
15	Finals Week			

J. Attendance Policy

Attendance is critical to successful learning. Absences related to jobs, health, and/or civil/military duties need to be made known to the instructor as early as possible. At least one full letter grade deduction will be applied **for 3 or more unexcused absences**.

K. Late Policy:

- Late submissions (in-class, project, assignment) will not receive credit.
- Makeup in-class tasks (and exams) are granted only for documented medical reasons or approved arrangements made at least 72 hours in advance.

L. Technical Skills and Technology required for the course:

- Students should be comfortable conducting online research, using standard office productivity tools, and installing or uninstalling software when required. Assistance will be provided as needed.
- Because class sessions will regularly include hands-on programming exercises and workshop activities, students are expected to bring a laptop to every class. Students who are unable to do so should contact the instructor in advance to discuss available options.

M. University Policy and Resources:

Visit the [Student Resources](#) site in Canvas for links to relevant university policies and resources, including information about academic support, student support, and library services.

N. Student Workload/Carnegie Hours

Activity	Instructor-led Weekly Avg.	Instructor-led Course Total	Independent Weekly Avg.	Independent Course Total
Lecture / Guided Programming / In- class Work	2.33	35.0	—	—
Exams	0.22	3.3	—	—
Project Workshops / Presentation	0.44	6.7	—	—
Assignments / Programming Exercises	—	—	2.50	37.5
Final Project	—	—	2.00	30.0
Reading / Course Preparation	—	—	1.50	22.5
Exam Preparation	—	—	1.50	22.5
Code Review / Testing / Documentation	—	—	0.50	7.5
Consultation with Instructor	—	—	0.00	0.0
TOTAL	3.00	45.0	8.00	120.0
TOTAL COURSE WORKLOAD				165 hours

Disclaimer

This syllabus may change from time to time to accommodate changing circumstances. Every effort will be made to alert students to changes that occur in a timely manner.